SysHealth — live demo runbook

Four EC2 sizes, one load, one verdict in dollars. Every command runs from `~/syshealth`.

1 The day before

- **Rehearse on the network you will present from.** The dashboard only lets in the address `./aws.sh up` ran from, on port 5000, and many campus networks block port 5000. Keep a phone hotspot as the way out.
- **Have the fallbacks ready:** the threshold screenshots, the README's results table, and `npm run dev` working on the laptop (section 5).
- **Do not run `./aws.sh deploy` on the day.** The fleet installs the code on GitHub, the version you rehearsed the day before.

2 Forty-five minutes before

The fleet needs about five minutes to boot and calibrate, and you want one rehearsal run in hand before anyone is watching.

Bring the fleet up

```
./aws.sh up --profile hackathon
```

It names the account and region, says what it costs and asks before building: about $0.19 an hour for the five machines, $0.22 with their addresses and disks.

Expect: `4 of 4 instances reporting`, then the **Dashboard** address and the **Token**. Copy both.

```
./aws.sh status
```

Expect: four sizes, `ONLINE yes`, all HEALTHY.

Rehearse once, from the dashboard

Open the Dashboard address. Press **Load test**, choose **Variable load** and **Ramp up**. The defaults are the test: your app at its busiest 1600 MB, from 200 to 3200 MB, 6 steps of 60 seconds. Press **Start on 4 instances** and paste the token into **Server token** when it asks. The browser keeps the token, so the live run will not ask again.

About seven minutes. It proves the whole chain works and leaves a finished result on the page in case the live run goes wrong. Then wait two minutes for the lines to fall back to zero. Lost the token, or Start says it is wrong? A rebuilt fleet has a new one:

```
(cd terraform && TF_WORKSPACE=fleet terraform output -raw token)
```

3 The demo, about twelve minutes

Step 1 — The problem (one minute, no clicking)

> **Say:** Teams pick an instance size by guessing, or by an alarm at 80 percent of memory in use. One way they overpay, the other way the app slows down and nobody knows why. SysHealth measures it: the same load on four sizes at the same moment, then the cheapest size that holds your app, in dollars a month.

Step 2 — The idle fleet

Four cards, all HEALTHY, a flat chart. Open **What the number means**.

> **Say:** This number comes from the Linux kernel itself: the share of time programs waited for memory. Not how full memory is. Under 10 percent is healthy, 10 to 40 is degraded, 40 or more is critical, and so is the whole machine stalling 10 percent of the time. Those lines come from systems that act on this signal in production: Android's low-memory killer, systemd-oomd and Meta's oomd. A state only changes after it has held for 15 seconds, so one spike is not a failure.

Step 3 — Start the ramp

Load test, **Variable load**, **Ramp up**, the same defaults, **Start on 4 instances**.

> **Say:** Same load, same moment, four machines. It climbs from 200 megabytes to 3.2 gigabytes over six minutes. Watch which ones feel it.

Step 4 — Talk through the six minutes

Minute	Load	Point at
1–2	800 MB	**t3.micro turns CRITICAL**: 13% waiting at 97% in use. Click it for **Where the memory is** and the likely cause.
2–3	1.4 GB	The kernel **kills the test on the micro**; its card counts the kills and it keeps reporting. **t3.small is 93% in use and still HEALTHY.**
3–4	2 GB	**t3.small turns CRITICAL**: 22% waiting.
4–6	2.6–3.2 GB	**t3.medium reaches 95% in use and stays HEALTHY.** t3.large is flat.

These are the 19 October numbers; a live run lands close to them. States judge a one-minute average, so a card turns up to a minute after its step begins. Use the quiet stretches for the Prometheus question (section 4).

> **Say:** At the 1.4 gigabyte step: look at the small. 93 percent of its memory is in use, so an 80 percent alarm would be ringing. But nothing is waiting, so nothing is slowed down. That is the difference between memory in use and memory pressure.

Step 5 — The verdict

Open **Load test**. **Which size to choose** leads the results.

> **Say:** Choose t3.medium, 32 dollars 70 a month. The small would save 16 dollars 35, but it holds only about 1.6 gigabytes, 5 percent more than the app needs, and we asked for 25. The large costs twice as much for room this app does not use. A sizing decision made from measurement, and said in money. Its last line says what it did not test: the same 4 gigabytes on Graviton or AMD, t4g.medium and t3a.medium, for about half. Those go in the next run before anyone switches.

Then point at the **Pressure at each load level** chart: where a size's line crosses the degraded line is the most it can hold. The table under it gives **Holds up to** and **Critical from** for each size. Change **Your app at its busiest**

or **Keep spare** and the answer is worked out again, without another test.

Step 6 — Beyond the dashboard (optional, one minute)

The same verdict from the terminal, instantly, from the test that just ran:

```
URL=http://<dashboard-ip>:5000      # ./aws.sh status prints it
python3 cli.py --server $URL report
```

> **Say:** The same test runs from the command line, so it can gate a build: it fails when the size the app needs has changed. Prometheus can scrape everything at /metrics, and an AI agent can run the experiment through an MCP server.

Name the gate rather than run it (seven minutes): `./aws.sh test --need 1600 --ramp 200:3200 --expect t3.medium` exits 3 when the answer changes, 2 when no size fits.

4 Questions you will get

“Why does the micro look calm at the end?”

> **Say:** It is not coping, it failed. The load did not fit in its memory and swap, so the kernel killed it. Pressure fell because the work stopped. The card counts the kills, and the results mark every step after it as critical.

Proof, live, if they want it: the kernel's own record, with times that match the chart.

```
./aws.sh ssh t3.micro 'sudo journalctl -k --since "20 min ago" | grep -i killed'
```

“Why did nothing show DEGRADED?”

> **Say:** On a real machine the change is sudden. Once memory and swap run out, the stall jumps past both lines within one step. Both sizes went critical through the whole-machine rule: the micro was fully stalled 12.9 percent of the time, the small 21.4. Degraded is for an app that is slowed but still working, and this load skipped it.

“How is this different from Prometheus and Grafana?”

> **Say:** Prometheus watches. It can graph this same kernel counter, but it never applies a load, never knows what load caused a reading, and never says what to buy. SysHealth runs the same load on every size at once, works out how much each holds before the app slows down, and names the cheapest that fits. They work together too: everything, the verdict included, is served at /metrics for Prometheus to scrape.

“Why not just watch memory in use?”

> **Say:** Because it does not say whether it hurts. At 95 percent in use the medium was stalled 0.03 percent of the time. At 97 percent the micro was stalled 13 percent. Pressure measures time lost to waiting, which is what users feel.

“A peak test said t3.small. Why does the ramp say t3.medium?”

> **Say:** A peak test only asks whether each size can hold the app at 1.6 gigabytes, not how much more it could take. The ramp measures the room left, and with 25 percent spare asked for, the small's 5 percent rules it out.

“It names t4g.medium at half the price. Why not pick that?”

> **Say:** It was not tested, and the verdict only picks what it measured. t3a.medium can join the next run with --types; Graviton is Arm, so it needs an Arm build first.

“Could this run in production?”

> **Say:** The measurement side, yes, and Prometheus can scrape it today. Sending samples and starting tests need a shared token, and the dashboard only admits one address. One thing would need work first: history is kept in memory, so a restart loses it and a real deployment would need a database.

“What is it built on?”

> **Say:** Python and Flask. The agent reads the kernel's pressure files every five seconds, the charts are hand-written SVG, and Terraform builds or destroys the fleet in one command.

5 If something breaks

The dashboard will not load

Almost always your address changed: a new network, or a mobile provider moving you overnight. Or the network blocks port 5000. It looks exactly like a dead server.

```
./aws.sh up --profile hackathon      # moves the firewall to this address
```

Expect: `0 added, 3 changed, 0 destroyed`. Still nothing: use the phone hotspot and run it again.

A phone on the laptop's Wi-Fi can open it too; on mobile data it is blocked.

A size is missing

```
./aws.sh status                      # which sizes are not reporting
./aws.sh ssh t3.small 'sudo tail -30 /var/log/syshealth-bootstrap.log'
./aws.sh ssh t3.small 'sudo journalctl -u syshealth -n 30'
```

Everything stays HEALTHY under load

The load fits. Check the **Right now** tile moves at all, then raise **To** in the form.

Nothing on AWS works and you must present anyway

A local simulation needs no AWS and no network: the real dashboard, a simulated fleet.

```
sudo apt install -y python3-flask python3-requests   # first time only
npm run dev                                          # http://127.0.0.1:5000
```

Say it is simulated (every page says **Demo data**), then show the README's real results.

6 Afterwards

Save the evidence first (history lives in the dashboard's memory), then destroy the fleet:

```
URL=http://<dashboard-ip>:5000
python3 cli.py --server $URL report --save rightsize-$(date +%F).json
curl -s "$URL/series?limit=720" > demo-evidence-$(date +%F).json
./aws.sh down && ./aws.sh status
```

Expect: `Nothing is up`: billing has stopped. Until then the fleet bills about $0.22 an hour, a little over $5 a day.